

JMeter API Testing & Automation Workshop — Complete Cheatsheet

Who this is for: You already know API testing in **Postman**. Now you're learning **Apache JMeter** — the same API testing skills, plus load and performance testing. Every step tells you **what** to do, **how** to do it, and **why** it matters. Where it helps, we bridge from a Postman concept you already know into its JMeter equivalent.

Postman bridge callouts look like this:  **From Postman:** In Postman you did X. In JMeter you do Y.

Table of Contents

1. [JMeter Installation & Launching](#)
2. [JMeter Fundamentals \(Architecture & Terminology\)](#)
3. [Your First JMeter Test](#)
4. [API Testing with JMeter \(GET/POST/PUT/PATCH/DELETE\)](#)
5. [Postman → JMeter Concept Mapping](#)
6. [Variables & Environment Configuration](#)
7. [Response Validation](#)
8. [Extracting Multiple Values](#)
9. [Request Chaining](#)
10. [JSR223 & Groovy](#)
11. [Dummy Sampler](#)
12. [Data-Driven Testing with CSV](#)
13. [Dynamic Data](#)
14. [Assertions](#)
15. [Controllers](#)
16. [Timers](#)
17. [Correlation](#)
18. [Load Testing Concepts](#)

19. [Load Test Example](#)
20. [Results & Reporting](#)
21. [HTML Dashboard Report](#)
22. [GUI vs Non-GUI Best Practices](#)
23. [Debugging & Troubleshooting](#)
24. [Complete Hands-On Mini Project](#)
25. [Quick Reference](#)

Practice APIs Used in This Workshop

We use free, public APIs so every example actually runs. Keep these in a browser tab.

API	Base URL	Used For
JSONPlaceholder	https://jsonplaceholder.typicode.com	Simple CRUD (GET/POST/PUT/PATCH/DELETE)
Restful-Booker	https://restful-booker.herokuapp.com	Auth tokens, chaining, the mini project
Reqres	https://reqres.in	Login → token, list users (needs header x-api-key: reqres-free-v1)
Google	https://www.google.com	Your very first (non-API) test

Note on Reqres: the free tier now requires the header x-api-key: reqres-free-v1. If a Reqres request returns 401, that header is missing.

Key Terms You'll Use All Day

Term	What It Means
Test Plan	The root of everything in JMeter — the whole .jmx file. Like a Postman Collection , but it also holds load configuration.

Term	What It Means
Thread Group	Defines the simulated users: how many, how fast they start, how many times they loop. This is your virtual user pool .
Thread / User	One simulated user. 50 threads = 50 users hitting the server in parallel.
Ramp-up	How long JMeter takes to start ALL the threads. 10 users over 10s = 1 user per second.
Loop Count	How many times each thread repeats its samplers.
Sampler	A single action that generates a request, e.g. HTTP Request . This is your Postman Request .
Listener	A component that collects and displays results (tables, graphs, trees).
Assertion	A pass/fail check on a response — status code, body text, response time, etc. Your Postman Tests .
Config Element	Reusable configuration shared by samplers: default host, headers, CSV data, variables.
Pre-Processor	Runs before a sampler (e.g. build a dynamic value). Like a Postman Pre-request Script .
Post-Processor	Runs after a sampler (e.g. extract a token from the response). Powers request chaining .
Timer	A pause added before samplers to simulate realistic user think-time.
Controller	Logic container that controls which samplers run and how (if/loop/once/transaction).
Extractor	A Post-Processor that pulls a value out of a response into a variable (JSON, Regex, XPath, Boundary).
.jmx	The XML file that stores your entire test plan. This is what you save, share, and version-control.
.jtl	The raw results file produced by a run (CSV or XML). Feeds reports and the HTML dashboard.

Term	What It Means
GUI mode	The visual JMeter app. For building and debugging tests.
Non-GUI (CLI) mode	Command-line execution (jmeter -n ...). For real load tests .
Throughput	Requests completed per second (or per minute). The server's processing rate.
Latency	Time from sending the request to receiving the first byte back.
Response Time	Time from sending the request to receiving the full response.
Percentile	"90% of responses were faster than X ms." Better than average for spotting slow outliers.
Correlation	Capturing a dynamic value from one response (a token, an ID) and feeding it into the next request.

Section 1 — JMeter Installation & Launching

What You'll Learn

How to install JMeter, launch it in several ways, open existing .jmx files, and understand the crucial split between GUI mode and Non-GUI mode.

Why This Matters

In Postman, "launching" was trivial — you opened an app. JMeter is a Java application with **two operating modes**, and picking the wrong one is the #1 beginner mistake. Running a real load test in GUI mode can make JMeter itself the bottleneck and give you wrong numbers. Getting launch and organization right from day one saves you from garbage results later.

How To Do It

1.1 — Prerequisite: Java

JMeter runs on Java. You need **Java 8 or newer** (Java 17+ recommended for current JMeter). Check it:

```
java -version
```

If you see a version number, you're good. If "command not found," install a JDK (Temurin/Adoptium, Oracle, or your OS package manager).

1.2 — Install JMeter

1. Download the binary ZIP from the Apache JMeter site (apache-jmeter-X.X.zip).
2. Unzip it anywhere, e.g. C:\apache-jmeter (Windows) or ~/apache-jmeter (Mac/Linux).
3. The important folder is bin/. Everything you launch lives there.

Optional but recommended — add JMeter to your PATH so you can type jmeter from any folder instead of navigating to bin/ every time. On Mac/Linux add export PATH=\$PATH:~/apache-jmeter/bin to your shell profile; on Windows add the bin folder to the system PATH environment variable.

1.3 — Launch the GUI

Go into the bin/ folder and run:

- **Windows:** double-click jmeter.bat (or run it from a terminal).
- **Mac/Linux:** run ./jmeter or sh jmeter.

A window opens with an empty **Test Plan** already created for you. You'll also see a console/terminal window — **do not close it**; that terminal is JMeter's engine. Closing it kills JMeter.

 **You will see this warning in the GUI:** *"Don't use GUI mode for load testing! only for Test creation and Test debugging."* JMeter itself is telling you the rule. Believe it.

1.4 — Launch from the command line (the ways that matter)

1. Just open the GUI (empty)

```
jmeter
```

2. Open the GUI with a specific test plan already loaded

```
jmeter -t test.jmx
```

3. Run a test in NON-GUI mode and write results to a .jtl file

```
jmeter -n -t test.jmx -l results.jtl
```

4. Run non-GUI AND generate an HTML dashboard afterward

```
jmeter -n -t test.jmx -l results.jtl -e -o report
```

5. Run non-GUI, passing a property in from the command line

```
jmeter -n -t test.jmx -l results.jtl -Jusers=100 -Jrampup=60
```

1.5 — Opening an existing .jmx file

Three ways, all valid:

1. **From the GUI:** File → Open → pick your .jmx.
2. **From the command line:** `jmeter -t path/to/test.jmx` (opens GUI with it loaded).
3. **Drag and drop:** drag a .jmx file straight onto the open JMeter window. It loads instantly.

1.6 — GUI vs Non-GUI mode — the core distinction

	GUI Mode	Non-GUI (CLI) Mode
Command	<code>jmeter</code>	<code>jmeter -n -t test.jmx -l results.jtl</code>
Purpose	Build & debug tests	Execute real load tests
Resource use	Heavy (rendering graphs live)	Light (headless)
User count	A handful, to verify logic	Hundreds / thousands
Results	Live on screen	Written to .jtl, analyzed after

The rule of thumb: *Build in GUI. Run for load in Non-GUI.* You develop and debug a plan with 1–2 threads in the GUI, watching the View Results Tree. Once it works, you close the GUI and fire the real test from the terminal with `-n`.

1.7 — Important command-line options (memorize these)

Flag	Meaning
-n	Non-GUI mode (headless run)
-t <file.jmx>	The test plan to run
-l <file.jtl>	Where to write raw results
-e	Generate report dashboard at end of run
-o <folder>	Output folder for the HTML dashboard (must be empty/non-existent)
-J<prop>=<val>	Set a JMeter property (read with <code>\${__P(prop)}</code>)
-G<prop>=<val>	Set a global property (passed to remote/distributed engines)
-r	Run remote (start all servers named in the properties file)
-R server1,server2	Run remote against specific servers
-H <host> -P <port>	Route traffic through a proxy
-	Force .jtl output format
Jmeter.save.saveservice.output_format=csv	

1.8 — How to save and organize test plans

- Save the plan with File → Save (or Ctrl+S). It writes a **.jmx** file.
- Give it a meaningful name: booking-api-smoke.jmx, checkout-load-100u.jmx.
- Keep a tidy project folder:

my-jmeter-project/

└─ test-plans/ # your .jmx files

└─ data/ # CSV data files

└─ results/ # .jtl output files

└─ reports/ # generated HTML dashboards

└─ lib/ # extra plugins / jars if needed

- .jmx is XML — it works beautifully in **Git**. Commit your test plans like source code.

Common Mistakes

- **Closing the black console window** → kills JMeter. Leave it open.
- **Running load in GUI mode** → skewed, slow, unreliable numbers.
- **Wrong Java version** → JMeter won't start or throws obscure errors. Check java - version first.
- **Reusing a non-empty -o report folder** → JMeter refuses to overwrite. Use a fresh folder each run.

Exercise 1

1. Launch JMeter in GUI mode.
2. From a terminal, run `jmeter -t` pointing at any .jmx you have (or create and save an empty one first).
3. Try dragging a .jmx onto the window.
4. Run `jmeter --help` (or `jmeter -?`) and skim the options list.

Section 2 — JMeter Fundamentals

What You'll Learn

The JMeter architecture and every core building block: what it is, why it's used, and a one-line example for each. This is your vocabulary for the rest of the workshop.

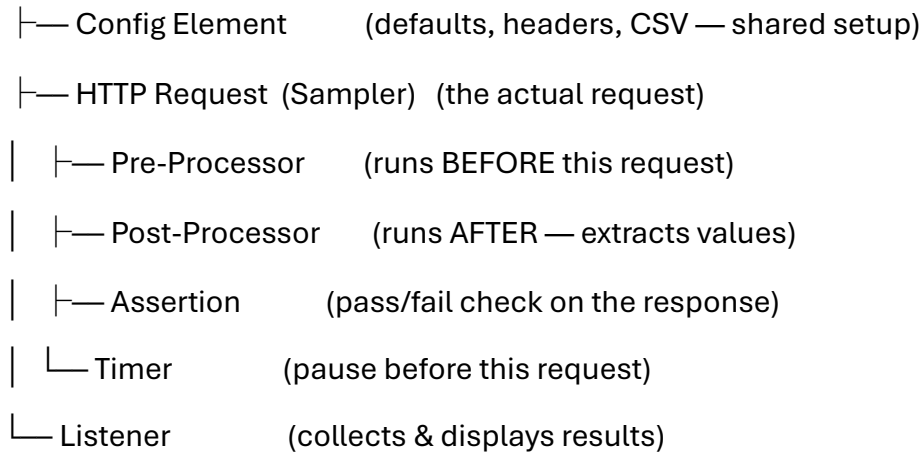
Why This Matters

Postman had a shallow object model: Collection → Request → Tests. JMeter has a **tree** of components that execute in a specific order. Once you understand the tree and the execution order, everything else clicks. Confusion about "why didn't my extractor run?" almost always comes from misunderstanding this hierarchy.

The Big Picture — JMeter's Tree

Test Plan (the whole project / .jmx)

└─ Thread Group (defines the virtual users)



Scope rule (important): a component's position determines what it affects. Put a Header Manager **directly under the Thread Group** → it applies to **all** samplers. Put it **under one HTTP Request** → it applies to **only that** sampler. This is JMeter's version of "collection-level vs request-level" settings in Postman.

Every Component Explained

Test Plan — *What:* the root node; the entire test. *Why:* holds global settings (user-defined variables, library paths, "Run thread groups consecutively"). *Example:* one .jmx file = one Test Plan named Booking API Tests.  **From Postman:** this is your **Collection** — but it also owns the load configuration.

Thread Group — *What:* defines your simulated users and how they run. *Why:* it's the engine of concurrency. *Example:* 50 threads, 10s ramp-up, loop 5 → 50 users start over 10 seconds and each runs the requests 5 times.  **From Postman:** this is your **Collection Runner**, but with real concurrency.

Threads / Users — *What:* the number of parallel virtual users. *Why:* concurrency is the whole point of load testing. *Example:* Number of Threads = 100 simulates 100 people at once.

Ramp-up period — *What:* seconds to start ALL threads. *Why:* starting 100 users in the same millisecond is unrealistic and can crash your own machine; ramping spreads them out. *Example:* 100 threads / 50s ramp-up = 2 new users per second.

Loop Count — *What:* how many times each thread repeats the samplers. *Why:* generate sustained traffic without adding users. *Example:* loop = 10 → each user sends the request block 10 times. Check "**Infinite**" to loop until stopped.

Duration & Scheduler — *What:* run for a fixed time rather than a fixed loop count. *Why:* endurance/soak tests run "for 1 hour" not "for N loops." *Example:* tick **Scheduler**, set Duration = 3600 (seconds) with Loop = Infinite → runs for exactly one hour.

Sampler — *What:* a component that sends one request and measures it. *Why:* it's the unit of work. The main one is **HTTP Request**; others include JDBC, FTP, JSR223, Debug, Dummy. *Example:* an HTTP Request sampler doing GET /booking/1.  **From Postman:** this is a **Request**.

HTTP Request — *What:* the sampler for web/API calls. *Why:* it's what you'll use 95% of the time. *Example:* method POST, path /auth, JSON body → returns a token.

Listener — *What:* collects results and shows them (tree, table, graph). *Why:* it's how you SEE what happened. *Example:* **View Results Tree** shows each request/response;

Summary Report shows aggregate stats.  **From Postman:** this is the **Console** + the runner results screen.

Assertion — *What:* a rule the response must satisfy. *Why:* turns "looks fine" into automated pass/fail. *Example:* Response Assertion checking the body contains "token".  **From Postman:** these are your pm.test(...) **Tests**.

Config Element — *What:* reusable configuration for samplers. *Why:* avoid repeating yourself (DRY). *Examples:* **HTTP Request Defaults** (set host once), **HTTP Header Manager** (set Content-Type once), **CSV Data Set Config** (feed test data), **User Defined Variables**.

 **From Postman:** this covers **Environments**, **collection variables**, and shared **Headers**.

Pre-Processor — *What:* runs just before a sampler. *Why:* prepare data (build a payload, set a variable) right before the request goes out. *Example:* a **JSR223 PreProcessor** that generates a fresh timestamp for the body.  **From Postman:** **Pre-request Script**.

Post-Processor — *What:* runs right after a sampler. *Why:* extract values from the response for later use — the heart of chaining/correlation. *Examples:* **JSON Extractor**, **Regular Expression Extractor**, **Boundary Extractor**, **JSR223 PostProcessor**.  **From Postman:** the extraction half of your **Tests** (pm.environment.set(...)).

Timer — *What:* a pause added before samplers in its scope. *Why:* real users don't fire requests back-to-back with zero gap. *Example:* **Constant Timer** of 1000ms adds a 1-second think-time before each request.

Controller — *What:* logic containers that decide which samplers run and how. *Why:* build conditional flows, grouping, and repetition. *Examples:* **If Controller** (run only if condition

true), **Loop Controller**, **Transaction Controller** (group requests into one timed unit), **Once Only Controller** (run login just once).

Execution Order (the rule that saves hours)

Within one scope, JMeter runs components in this order:

1. Config elements (set up)
2. Pre-Processors
3. Timers
4. Sampler (the request fires)
5. Post-Processors (extract values)
6. Assertions (validate)
7. Listeners (record results)

Why this matters: if your extractor "isn't working," check it's a **Post**-Processor attached to the **right sampler** — it can only run *after* that sampler and only sees *that* sampler's response.

Exercise 2

Draw the tree for this scenario on paper (no JMeter yet):

"20 users, ramping over 20 seconds, each logs in once, then reads their profile 3 times, validating a 200 each time, with a 1-second pause between reads, and all results recorded in a table."

Label which node is the Thread Group, which are Samplers, where the Timer goes, where the Assertion goes, and which Controller makes the login run only once.

Section 3 — Your First JMeter Test

What You'll Learn

Build a complete working test from an empty Test Plan against <https://www.google.com>:
Thread Group → HTTP Request → Assertion → Listener → Run → read the numbers.

Why This Matters

This is your /healthcheck moment. Once you can build, run, and read one test, every other test is a variation. We use Google here (not an API) so you focus purely on the JMeter mechanics before adding API complexity.

How To Do It

3.1 — Add a Thread Group

1. Right-click **Test Plan** → **Add** → **Threads (Users)** → **Thread Group**.
2. Set:
 - **Number of Threads (users):** 1
 - **Ramp-up period (seconds):** 1
 - **Loop Count:** 1
3. Name it TG - Google Smoke.

Start with **1 user, 1 loop**. You're verifying the test works, not loading anything yet.

3.2 — Add an HTTP Request sampler

1. Right-click **Thread Group** → **Add** → **Sampler** → **HTTP Request**.
2. Set:
 - **Protocol:** https
 - **Server Name or IP:** www.google.com
 - **Method:** GET
 - **Path:** /
3. Name it HTTP - GET Google Home.

Notice the split fields. Unlike Postman's single URL bar, JMeter splits the URL into Protocol / Server / Port / Path. This is deliberate — it lets you change the host in one place (via HTTP Request Defaults) without editing every sampler.

3.3 — Add an Assertion

1. Right-click the HTTP Request → **Add** → **Assertions** → **Response Assertion**.
2. Configure:
 - **Field to Test:** Response Code

- **Pattern Matching Rules:** Equals
- Click **Add** → enter 200.

This is your first automated check: the request must return HTTP 200.

3.4 — Add Listeners

1. Right-click **Thread Group** → **Add** → **Listener** → **View Results Tree**.
2. Right-click **Thread Group** → **Add** → **Listener** → **Summary Report**.

View Results Tree shows each individual request/response (your debugging window).

Summary Report shows aggregate numbers.

3.5 — Save and Run

1. Save the plan: Ctrl+S → google-smoke.jmx.
2. Click the green **Start** (▶) button in the toolbar.

3.6 — Read the results

Open **View Results Tree**:

- The sampler should be **green** . Red means failure.
- Click it → tabs on the right: **Sampler result** (status code, latency, size), **Request** (what was sent), **Response data** (the HTML that came back).

Open **Summary Report** and read the columns:

Column	Meaning
# Samples	How many requests were sent
Average	Mean response time (ms)
Min / Max	Fastest / slowest response
Std. Dev.	Spread of response times (consistency)
Error %	Percentage of failed requests
Throughput	Requests completed per second
Received KB/sec	Data download rate

3.7 — Now scale it up: 1 user vs 10 users

Go back to the Thread Group and change:

- **Number of Threads:** 10
- **Ramp-up period:** 10
- **Loop Count:** 1

Run again. Now interpret:

- **10 users, ramp-up 10s** → JMeter starts one new user each second. By second 10, all 10 are active.
- **Loop count 1** → each user hits Google once. So **10 total requests** (10 users × 1 loop).
- If you set **Loop = 5**, you'd get **50 requests** (10 × 5).

3.8 — Understand the key metrics

- **Requests per second (Throughput):** if 10 requests complete in ~2 seconds, throughput ≈ 5/sec. It's the server's *processing rate*.
- **Response time:** how long each individual request took. Watch **Average** and **Max**.
- **Error %:** should be 0%. Anything above 0 means failures — investigate in View Results Tree.
- **Throughput vs users:** more users usually means higher throughput — until the server saturates, at which point response time climbs and throughput flattens. Finding that point is the goal of load testing (Section 18).

Mental model of ramp-up: ramp-up is the *on-ramp to a highway*. 100 cars merging one-per-second is smooth; 100 cars appearing at once is a pile-up. Ramp-up spreads the load's start so it's realistic and doesn't overwhelm your own machine.

Checkpoint

You built a Test Plan from scratch: Thread Group → HTTP Request → Assertion → 2 Listeners, ran it with 1 user then 10, and read throughput, response time, and error %. Every test in this workshop is this same skeleton with more detail.

Common Mistakes

- **No listener added** → you run the test and see nothing. Always add View Results Tree while building.
- **Typing the full URL in Server Name** (https://www.google.com) → JMeter treats the whole thing as a hostname and fails. Protocol goes in its own field; Server Name is just www.google.com.
- **Forgetting the leading / in Path.**

Exercise 3

1. Change to **5 users, ramp-up 5s, loop 3**. Predict the total number of requests before running. (Answer: 15.)
2. Add a second HTTP Request to /search?q=jmeter and re-run. Watch both samplers in the tree.
3. Break the assertion on purpose (expect 404) and confirm the sampler turns red.

Section 4 — API Testing with JMeter

What You'll Learn

Move from "hit a website" to real API testing: all HTTP methods, query/path parameters, headers, JSON bodies, content types, and authentication (Bearer tokens, Authorization headers).

Why This Matters

This is where your Postman knowledge maps almost 1:1. You already understand GET/POST/PUT/PATCH/DELETE and status codes. Here you learn the JMeter *mechanics* of expressing them — mainly the **Body Data** tab and the **HTTP Header Manager**.

Setup: HTTP Request Defaults (do this once)

So you don't retype the host in every sampler:

1. Right-click **Thread Group** → **Add** → **Config Element** → **HTTP Request Defaults**.
2. Set **Protocol**: https, **Server Name**: jsonplaceholder.typicode.com.

Now every HTTP Request in this Thread Group can leave Protocol/Server blank and just specify the Path.  **From Postman**: this is your `{{baseUrl}}` — set once, used everywhere.

Setup: HTTP Header Manager (do this once)

1. Right-click **Thread Group** → **Add** → **Config Element** → **HTTP Header Manager**.
2. Add a header: **Name** Content-Type, **Value** application/json.
3. Add another: **Name** Accept, **Value** application/json.

 **From Postman:** these are the headers you set on the request, but here they're shared across all samplers in scope.

4.1 — GET request

Sampler: **Method** GET, **Path** /posts/1. Expected: 200 OK with a JSON post object.

4.2 — GET with query parameters

Two ways:

- **Path with query string:** /posts?userId=1
- **Parameters table (cleaner):** Path /posts, then in the **Parameters** tab add row:
Name userId, Value 1. JMeter builds /posts?userId=1 for you.

 **From Postman:** the Parameters tab = Postman's **Params** tab.

4.3 — Path parameters

There's no special "path variable" field — you build it into the Path using a JMeter variable:

- Path: /posts/\${postId} where \${postId} is defined elsewhere (User Defined Variables, CSV, or an extractor). Expected for /posts/1 → 200.

4.4 — POST with a JSON body

1. New HTTP Request → **Method** POST, **Path** /posts.
2. Click the **Body Data** tab (NOT Parameters).
3. Paste:

```
{  
  "title": "JMeter Rocks",  
  "body": "Learning API testing",  
  "userId": 1  
}
```

Expected: 201 Created, and the response echoes your object with a new id.

Body Data vs Parameters — the key POST decision.

- **Body Data** → raw JSON payload (what you want for REST APIs). Requires Content-Type: application/json (from your Header Manager).
- **Parameters** → form-encoded key/value pairs (application/x-www-form-urlencoded). For JSON APIs, always use **Body Data**.

 **From Postman:** Body Data (raw) = Postman's **Body** → raw → JSON.

4.5 — PUT (full replacement)

Method PUT, **Path** /posts/1, **Body Data**:

```
{  
  "id": 1,  
  "title": "Fully Replaced",  
  "body": "All fields sent",  
  "userId": 1  
}
```

Expected: 200 OK. PUT sends the complete resource.

4.6 — PATCH (partial update)

Method PATCH, **Path** /posts/1, **Body Data**:

```
{ "title": "Only the title changed" }
```

Expected: 200 OK. Only the fields you send change.

If the PATCH method isn't in the dropdown in an old JMeter version, type it in — the method field accepts custom verbs.

4.7 — DELETE

Method DELETE, **Path** /posts/1. Usually no body. Expected: 200 OK (JSONPlaceholder returns {}).

4.8 — Authentication & Bearer tokens

Most real APIs need auth. You express it as a **header** in the HTTP Header Manager.

Bearer token (most common):

- Header **Name:** Authorization
- Header **Value:** Bearer \${authToken}

where \${authToken} is a variable holding a token you extracted from a login response (Section 9 shows how to get it).

Basic auth:

- Header **Name:** Authorization
- Header **Value:** Basic YWRtaW46cGFzc3dvcmQxMjM= (base64 of user:pass)
- Or use the **HTTP Authorization Manager** config element, which builds the header for you.

Cookie-based token (Restful-Booker uses this for PUT/DELETE):

- Header **Name:** Cookie
- Header **Value:** token=\${authToken}



From Postman: the **Authorization** tab in Postman generates a header behind the scenes. In JMeter you set that header yourself — which makes the mechanism explicit and easy to debug.

4.9 — Content-Type & Authorization together

A typical authenticated JSON request needs these headers (put them in the Header Manager):

Name	Value
Content-Type	application/json
Accept	application/json
Authorization	Bearer \${authToken}

Realistic practice set (run these now)

#	Method	URL	Body	Expect
1	GET	jsonplaceholder.typicode.com/users	—	200, array of 10
2	GET	.../users/2	—	200, one user

#	Method	URL	Body	Expect
3	GET	.../posts?userId=1	—	200, filtered array
4	POST	.../posts	JSON above	201
5	PUT	.../posts/1	full JSON	200
6	PATCH	.../posts/1	partial JSON	200
7	DELETE	.../posts/1	—	200

Common Mistakes

- **Putting JSON in the Parameters tab** instead of Body Data → server gets form-encoded junk → 400/415.
- **Missing Content-Type: application/json** → API can't parse the body → 400/415.
- **Full URL in Server Name field** → connection error.
- **Forgetting the Bearer prefix** (with the trailing space) in the Authorization header.

Exercise 4

1. Build all 7 requests above inside one Thread Group using HTTP Request Defaults + Header Manager.
2. Add a Response Assertion to each checking the correct status code.
3. Switch the POST body's Content-Type header off (disable the Header Manager) and observe what changes.

Section 5 — Postman → JMeter Concept Mapping

What You'll Learn

A complete translation table from the Postman world you know into the JMeter world you're learning. Keep this open as a "dictionary" for the rest of the workshop.

Why This Matters

Every hour you spent in Postman is transferable — you just need the JMeter word for each idea. This table is the single fastest way to feel at home in JMeter.

The Master Mapping Table

Postman	JMeter	Notes
Collection	Test Plan (.jmx)	The whole project/file.
Folder	Simple Controller / Transaction Controller	Group related samplers.
Request	HTTP Request (Sampler)	The core request unit.
URL bar (single field)	Protocol + Server + Port + Path (split fields)	JMeter splits the URL.
{{baseUrl}}	HTTP Request Defaults + \${baseUrl} variable	Set host once, reuse everywhere.
Params tab	Parameters tab	Query-string key/values.
Body → raw → JSON	Body Data tab	Raw JSON payload.
Headers tab	HTTP Header Manager	Shared or per-sampler headers.
Authorization tab	HTTP Header Manager / HTTP Authorization Manager	You set the header explicitly.
Environment	User Defined Variables / CSV Data Set / .properties file	Config that varies per environment.
Environment / Collection Variable	JMeter Variable (\${var}) or Property (\${__P(var)})	Vars are per-thread; props are global.
{{variable}} syntax	\${variable} syntax	Different braces, same idea.
Pre-request Script	JSR223 PreProcessor (Groovy)	Runs before the request.
Tests / pm.test(...)	Assertions + JSR223 PostProcessor	Validation after the response.

Postman	JMeter	Notes
pm.expect(...).to.eql(...)	Response Assertion / JSON Assertion / JSR223 Assertion	Different assertion elements.
pm.environment.set("t", val)	JSON Extractor / Regex Extractor / vars.put("t", val)	Save a value for later.
pm.environment.get("t")	\${t} or vars.get("t")	Read a saved value.
pm.response.json()	prev.getResponseDataAsString() / extractors	Access the response body.
Request chaining	Post-Processor Extractors + Variables	Token/ID from one request into the next.
Collection Runner	Thread Group	Runs the flow; adds concurrency.
Iterations (Runner)	Loop Count / Loop Controller	Repeat the flow.
Data file (CSV in Runner)	CSV Data Set Config	Feed rows of test data.
pm.iterationData.get("col")	\${col} (from CSV Data Set Config)	Access current data row.
Console (console.log)	log.info(...) + View Results Tree	Debug output.
Test Results tab	Assertion results in View Results Tree / Assertion Results listener	Pass/fail display.
Newman (CLI runner)	jmeter -n -t ... -l ...	Headless execution.
Newman HTML report	JMeter HTML Dashboard (-e -o)	Shareable report.
Postman monitors	Scheduled JMeter runs (CI/cron)	Recurring execution.
Mock server	Dummy Sampler	Fake a response for testing flow.
(No direct equivalent)	Timers	Simulate user think-time — a load-testing

Postman	JMeter	Notes
		concept Postman lacks.
(No direct equivalent)	Ramp-up / Threads / Throughput Timer	Concurrency & load shaping.

The One Big Mental Shift

Postman answers "does this endpoint work?" JMeter also answers "does it work when 500 people hit it at once?" Everything you knew about *functional* API testing carries over. JMeter adds a second dimension — **concurrency and performance** — that Postman never had. Sections 1–17 cover the functional side you already understand; Sections 18–21 are the new performance dimension.

Exercise 5

Take three things you did in your Postman workshop — for example (1) setting `{{baseUrl}}`, (2) a pre-request script, (3) `pm.environment.set("token", ...)` — and, using the table, write down the JMeter component you'd use for each. You'll build all three in the next sections.

Section 6 — Variables & Environment Configuration

What You'll Learn

How to stop hardcoding values: User Defined Variables, the `${...}` syntax, JMeter properties, command-line `-J` overrides, and per-environment (DEV/QA/PROD) configuration.

Why This Matters

In Postman you never hardcoded `http://localhost:8000` — you used `{{baseUrl}}`. Same discipline here. Hardcoded hosts, tokens, and IDs make a test plan unusable across environments and impossible to maintain. JMeter gives you several layers of configuration; knowing which to use when is a core skill.

Variables vs Properties — the crucial distinction

Variables	Properties
Syntax <code>\${myVar}</code>	<code>\$_P(myProp)</code>

	Variables	Properties
Scope	Per-thread (each user has its own copy)	Global (shared across all threads)
Set by	User Defined Variables, extractors, vars.put()	-J on CLI, .properties file, props.put()
Use for	Per-user data, extracted tokens, IDs	Environment switches, run parameters (users, host)

 **From Postman:** a **variable** is like an environment variable scoped to one runner iteration; a **property** is like a global you pass in at launch time.

6.1 — User Defined Variables (UDV)

1. Right-click **Test Plan** → **Add** → **Config Element** → **User Defined Variables**.
2. Add rows:

Name	Value
baseUrl	restful-booker.herokuapp.com
protocol	https
username	admin
password	password123

3. Use them anywhere with `${baseUrl}`, `${username}`, etc.

Now set **HTTP Request Defaults** → Server Name = `${baseUrl}`, Protocol = `${protocol}`. Your host is defined in exactly one place.

6.2 — The `${variable}` syntax

- Reference any variable: `${authToken}`, `${userId}`.
- Works in URLs, paths, bodies, headers, assertions — anywhere text is accepted.
- In Body Data:

```
{ "username": "${username}", "password": "${password}" }
```

6.3 — Environment-specific config (DEV / QA / PROD)

Approach A — Properties files (recommended). Create one file per environment:

dev.properties

baseUrl=dev-api.example.com

port=8080

token=dev-token-123

qa.properties

baseUrl=qa-api.example.com

port=443

token=qa-token-456

Load the right file at launch:

```
jmeter -n -t test.jmx -q dev.properties -l results.jtl
```

Read a property inside the plan with `${__P(baseUrl)}`. Provide a fallback default:

`${__P(baseUrl,localhost)}` → uses localhost if the property isn't set.

Approach B — a single `${env}` switch. Put a UDV `env=qa`, then have a CSV or if logic pick the matching host. Approach A is cleaner for most teams.

6.4 — Command-line properties: `-Jproperty=value`

Override anything at launch without editing the .jmx:

```
jmeter -n -t test.jmx -l results.jtl -JbaseUrl=qa-api.example.com -Jusers=100 -Jrampup=60
```

Inside the plan, read them:

- Thread Group → Number of Threads: `${__P(users,10)}`
- Thread Group → Ramp-up: `${__P(rampup,10)}`
- Host: `${__P(baseUrl,localhost)}`

This is the professional pattern: **one .jmx, many environments and load levels, all controlled from the command line.** The same test plan runs a 10-user smoke test in DEV and a 500-user load test in QA — you just change the -J flags.

6.5 — Using variables across requests

Variables set by an extractor in one sampler are available (as `${var}`) in every following sampler **in the same thread**. This is exactly how token chaining works (Section 9). Example flow:

POST /auth → JSON Extractor saves `${authToken}`

PUT /booking/1 → header: Cookie: token=`${authToken}`

Common Mistakes

- **Using `${__P(x)}` for a per-user extracted value** → properties are global; a token extracted per user must be a **variable** (`${x}`), not a property. Mixing these causes users to share/overwrite each other's tokens.
- **Forgetting the default in `${__P(name,default)}`** → if the property is missing, JMeter substitutes the literal string `${__P(name)}` and requests break confusingly.
- **Hardcoding the host in each sampler** instead of HTTP Request Defaults.

Exercise 6

1. Create UDV's for baseUrl, username, password.
2. Wire HTTP Request Defaults to `${baseUrl}`.
3. Make Number of Threads read `${__P(users,5)}`.
4. Run once from GUI (should default to 5 users). Then run from CLI with `-Jusers=20` and confirm it uses 20.

Section 7 — Response Validation

What You'll Learn

Validate far beyond "is it 200?" — response codes, body text, JSON fields, headers, response time — and the four extractor types that pull values out of responses.

Why This Matters

Your Postman workshop hammered this point: a QA's job is finding how things break, not just confirming they work. A test that only checks status code passes even when the API returns garbage in the body. Real validation checks the *content*.

7.1 — Response code validation

Response Assertion → Field to Test: **Response Code** → Equals → 200. For a range (any 2xx):
Field to Test **Response Code**, Pattern Matching **Matches**, pattern 2\d\d.

7.2 — Response body / text validation

Response Assertion → Field to Test: **Text Response** → **Contains** → pattern "firstname".

- **Contains** — substring/regex match anywhere in the body.
- **Matches** — the whole response matches the regex.
- **Equals** — exact string match.
- **Substring** — plain (non-regex) substring. Tick **Not** to assert something is *absent* (e.g. body must NOT contain error).

7.3 — JSON validation

Use the **JSON Assertion** (validate a JSON path's value directly):

- **Assert JSON Path exists:** \$.firstname
- **Additionally assert value:** tick "Additionally assert value" → Expected Value Jim.

Example paths for a booking response:

- \$.bookingid — the ID exists
- \$.booking.firstname — nested field
- \$.booking.bookingdates.checkin — deeply nested

7.4 — Header validation

Response Assertion → Field to Test: **Response Headers** → Contains → Content-Type: application/json.

7.5 — Response time validation

Duration Assertion → **Duration in milliseconds** = 2000. The sampler fails if it took longer than 2s. This is your performance gate baked into a functional test.  **From Postman:**
equals pm.expect(pm.response.responseTime).to.be.below(2000).

7.6 — The four Extractors (Post-Processors)

Extractors pull a value from a response into a variable. Choose by response format:

Extractor	Best for	Example config
JSON Extractor	JSON responses (most APIs)	JSON Path \$.token → var authToken
Regular Expression Extractor	Any text, non-JSON, or tricky cases	Regex "token":"([^\"]+)" → template \$1\$
Boundary Extractor	Value between a known left & right string	Left "token:" Right " → var authToken
XPath2 Extractor	XML / HTML responses	XPath //book/title/text()

7.7 — The core pattern: extract → store → reuse → validate

This is the loop your whole automation depends on:

1. **Extract** a value from response A with a Post-Processor.
2. **Store** it in a variable, e.g. \${authToken}.
3. **Use** it in request B (header, path, or body).
4. **Validate** later that the value did what you expected.

Worked example (Restful-Booker):

Request: POST /auth with body {"username":"admin","password":"password123"} Add a JSON Extractor to it:

- Names of created variables: authToken
- JSON Path expressions: \$.token
- Match No.: 1

Now \${authToken} holds the token. Add a Response Assertion to confirm extraction worked:

- Field: **JMeter Variable** → variable name authToken → Matches → \w+ (non-empty word).

Use it in PUT /booking/1 header: Cookie: token=\${authToken}.

7.8 — JMESPath (JSON JMESPath Extractor)

Newer JMeter ships a **JSON JMESPath Extractor** (and matching assertion). JMESPath is more powerful than plain JSON Path for filtering/transforming:

- bookingdates.checkin — nested access
- booking.firstname — dotted path
- [?totalprice > \100`].bookingid` — filter expressions Use it when JSON Path can't express the query you need.

Common Mistakes

- **Extractor attached to the wrong sampler** → it only sees the response of the sampler it's nested under.
- **Match No. = 0** means "random match"; use 1 for the first match, -1 for all matches.
- **Asserting only the status code** → your Postman instincts already warn you against this. Check the body too.

Exercise 7

For GET <https://restful-booker.herokuapp.com/booking/1>:

1. Add a JSON Assertion that \$.firstname exists.
2. Add a Duration Assertion of 3000ms.
3. Add a JSON Extractor pulling \$.lastname into \${lastName}.
4. Add a Response Assertion on the **JMeter Variable** lastName confirming it's non-empty.

Section 8 — Extracting Multiple Values

What You'll Learn

Pull several fields out of one response into several variables, then reuse and debug them.

Why This Matters

Real responses aren't one value — a login might return a token, a user id, a role, and an expiry. You need all of them downstream. Extracting many values cleanly (and verifying you got them) is an everyday task.

The response we'll work with

```
{  
  "id": 123,  
  "name": "John",  
  "status": "ACTIVE",  
  "token": "abc123"  
}
```

Goal: extract id, name, status, token into four variables.

8.1 — One JSON Extractor, multiple variables

The **JSON Extractor** accepts multiple fields separated by semicolons — the three lists must line up positionally.

- **Names of created variables:** userId;userName;userStatus;userToken
- **JSON Path expressions:** \$.id;\$.name;\$.status;\$.token
- **Match No.s:** 1;1;1;1
- **Default Values:** NOT_FOUND;NOT_FOUND;NOT_FOUND;NOT_FOUND

After this runs you have \${userId}=123, \${userName}=John, \${userStatus}=ACTIVE, \${userToken}=abc123.

The semicolon-aligned lists are the single most useful JSON Extractor trick. One extractor, four variables.

8.2 — Alternative: separate extractors

You can also add four JSON Extractors, one per field. More clicks, but clearer for beginners and easier to enable/disable individually.

8.3 — Alternative: JSR223 PostProcessor (full control)

When logic gets complex, parse once in Groovy and set everything:

```
import groovy.json.JsonSlurper  
  
def json = new JsonSlurper().parseText(prev.getResponseDataAsString())  
  
vars.put("userId", json.id.toString())  
  
vars.put("userName", json.name)
```

```
vars.put("userStatus", json.status)
```

```
vars.put("userToken", json.token)
```

```
log.info("Extracted user: " + json.name + " token: " + json.token)
```

prev = the previous sampler's result; vars = the variables store. (More on these in Section 10.)

8.4 — Reuse the variables downstream

Later samplers use them freely:

- Path: /users/\${userId}
- Header: Authorization: Bearer \${userToken}
- Body: { "owner": "\${userName}", "state": "\${userStatus}" }

8.5 — Inspecting / debugging extracted variables

Three reliable ways:

1. **Debug Sampler** — right-click Thread Group → **Add** → **Sampler** → **Debug Sampler**. When you run, its "Response data" in View Results Tree lists **every JMeter variable and its current value**. This is your #1 debugging tool for "did my extraction work?"
2. **log.info() in a JSR223 element** — prints to the JMeter console.
3. **Dummy Sampler** (Section 11) — echo the variables into a fake response you can eyeball.

Add a Debug Sampler right after any extractor while building. It instantly answers "is \${userToken} actually set, or empty?"

Common Mistakes

- **Mismatched list lengths** — 4 variable names but 3 JSON paths → misaligned/blank results.
- **Reading a variable before it's extracted** — order in the tree matters; the extractor's sampler must run first.
- **Forgetting Default Values** — set them so a failed extraction shows an obvious NOT_FOUND instead of an empty string that silently breaks the next request.

Exercise 8

1. Send POST `https://reqres.in/api/login` (header `x-api-key: reqres-free-v1`) with body `{"email":"eve.holt@reqres.in","password":"cityslicka"}`.
 2. Extract `$.token` into `${authToken}`.
 3. POST `https://reqres.in/api/users` and extract **both** `$.id` and `$.createdAt` in one JSON Extractor.
 4. Add a Debug Sampler and confirm all three variables are populated.
-

Section 9 — Request Chaining

What You'll Learn

Wire multiple requests into a dependent flow where each step uses values produced by earlier steps — the pattern behind almost every real API test.

Why This Matters

You can't create a booking without logging in first; you can't fetch a user without the id you got when you created them. Chaining is how you model real workflows. In Postman you did this with `pm.environment.set` in Tests + `{{var}}` in the next request. JMeter does it with **Post-Processor extractors + `${var}`**.

The chain we'll build (Restful-Booker)

POST `/auth` → extract `${authToken}`

↓

POST `/booking` → extract `${bookingId}`

↓

GET `/booking/${id}` → validate the data matches

↓

PUT `/booking/${id}` → uses `${authToken}` to update

↓

DELETE `/booking/${id}` → uses `${authToken}` to clean up

9.1 — Step 1: Login and extract the token

Sampler POST `/auth`:

- Path: /auth, Body Data:

```
{ "username": "admin", "password": "password123" }
```

- Add **JSON Extractor**: variable authToken, JSON Path \$.token, Match No. 1.
- Add **Response Assertion**: Response Code Equals 200.

9.2 — Step 2: Create a booking and extract the id

Sampler POST /booking:

- Path: /booking, Body Data:

```
{  
  "firstname": "Jim",  
  "lastname": "Brown",  
  "totalprice": 111,  
  "depositpaid": true,  
  "bookingdates": { "checkin": "2024-01-01", "checkout": "2024-01-05" },  
  "additionalneeds": "Breakfast"  
}
```

- Add **JSON Extractor**: variable bookingId, JSON Path \$.bookingid, Match No. 1.
- Assertion: Response Code Equals 200 (Booker returns 200 on create).

9.3 — Step 3: Get the booking and validate

Sampler GET /booking/\${bookingId}:

- Path: /booking/\${bookingId}
- **JSON Assertion**: \$.firstname equals Jim.
- **JSON Assertion**: \$.totalprice equals 111.

This proves the record we created is really there with the right data.

9.4 — Step 4: Update (needs the token)

Sampler PUT /booking/\${bookingId}:

- Add an **HTTP Header Manager** under this sampler with Cookie: token=\${authToken} and Content-Type: application/json.
- Body Data: same shape as create but change firstname to James.
- Assertion: Response Code Equals 200; JSON Assertion \$.firstname = James.

9.5 — Step 5: Delete (needs the token)

Sampler DELETE /booking/\${bookingId}:

- Header Manager: Cookie: token=\${authToken}.
- Assertion: Response Code Equals 201 (Booker returns 201 on delete).

9.6 — Why Post-Processors make this work

Each extractor runs **after** its sampler and writes into the **thread's** variable store. Because all samplers in a Thread Group run top-to-bottom in the same thread, a variable set in step 1 is visible in steps 2–5. That thread-scoped visibility is the entire mechanism.

Add a Debug Sampler between steps while building so you can see \${authToken} and \${bookingId} populate as the chain progresses.

Common Mistakes

- **Auth header missing on PUT/DELETE** → Booker returns 403 Forbidden. The token must be in the Cookie header.
- **Using \${bookingId} before Step 2 runs** → empty path /booking/ → 404.
- **A shared Header Manager overriding a per-sampler one** — remember scope: a Header Manager under the Thread Group applies everywhere; add auth headers under the specific sampler that needs them (or manage carefully).

Exercise 9

Build the full 5-step chain above. Then reorder DELETE before GET and observe the GET now 404s — proof that chain order is a real dependency, not decoration.

Section 10 — JSR223 & Groovy

What You'll Learn

JMeter's scripting layer: the JSR223 Sampler / PreProcessor / PostProcessor / Assertion, the Groovy language basics, and the built-in objects vars, props, ctx, prev, log.

Why This Matters

This is the direct heir to your Postman **Pre-request Scripts** and **Tests**. When built-in elements can't do what you need — generate dynamic data, transform a value, do custom validation — you drop into Groovy. It's the most powerful tool in JMeter.

What "JSR223" means

JSR223 is just the Java standard for embedding scripts. In JMeter you'll always choose **Groovy** as the language (it's fast and pre-installed). There are four JSR223 flavors:

Element	Runs	Postman equivalent
JSR223 Sampler	As its own request/action	(a script step)
JSR223 PreProcessor	Before a sampler	Pre-request Script
JSR223 PostProcessor	After a sampler	extraction part of Tests
JSR223 Assertion	After a sampler, to pass/fail <code>pm.test(...)</code>	

Always pick Groovy in the Language dropdown, and (for repeated runs) tick **"Cache compiled script if available"** for speed.

The built-in objects (memorize these five)

Object	What it is	Common use
vars	The thread's variables (JMeterVariables)	<code>vars.get("x"), vars.put("x","1")</code>
props	Global properties	<code>props.get("baseUrl"), props.put("k","v")</code>
ctx	The thread context	<code>ctx.getThreadNum()</code> , engine internals
prev	The previous SampleResult	<code>prev.getResponseDataAsString(), prev.getResponseCode()</code>
log	The logger	<code>log.info("msg"), log.error("msg")</code>

 **From Postman:** `vars` $\approx$ `pm.environment`, `prev` $\approx$ `pm.response`, `log` $\approx$ `console.log`.

10.1 — Generate dynamic data (PreProcessor)

```
// Build a unique email before a registration request
```

```
def unique = System.currentTimeMillis()
vars.put("email", "user_" + unique + "@test.com")
log.info("Generated email: " + vars.get("email"))
```

Use \${email} in the request body.

10.2 — Manipulate variables

```
def price = vars.get("totalprice").toInteger()
vars.put("discountedPrice", (price * 0.9).toInteger().toString())
```

10.3 — Read response data (PostProcessor)

```
import groovy.json.JsonSlurper
def body = prev.getResponseDataAsString()
def json = new JsonSlurper().parseText(body)
vars.put("bookingId", json.bookingid.toString())
log.info("Status: " + prev.getResponseCode() + " Booking: " + json.bookingid)
```

10.4 — Extract a custom value

```
// Grab the 2nd word of a plain-text response
def parts = prev.getResponseDataAsString().split(" ")
vars.put("secondWord", parts.length > 1 ? parts[1] : "N/A")
```

10.5 — Custom validation (JSR223 Assertion)

```
import groovy.json.JsonSlurper
def json = new JsonSlurper().parseText(prev.getResponseDataAsString())

if (json.totalprice == null || json.totalprice <= 0) {
    AssertionResult.setFailure(true)
    AssertionResult.setFailureMessage("totalprice invalid: " + json.totalprice)
}
```

AssertionResult is a special object available only in the JSR223 **Assertion** — set failure true to fail the sample.

10.6 — Generate timestamps

```
def now = new Date()

vars.put("today", now.format("yyyy-MM-dd"))

vars.put("epoch", (now.time / 1000L).toString())
```

10.7 — Create random values

```
def rnd = new Random()

vars.put("randomAge", (rnd.nextInt(80) + 18).toString()) // 18..97

vars.put("randomId", UUID.randomUUID().toString())
```

10.8 — Log information

```
log.info("=== Thread " + ctx.getThreadNum() + " sending booking ===")

log.warn("Token was empty!") // shows in jmeter.log and the console
```

10.9 — Pass data between requests

Anything you `vars.put(...)` is readable as `${...}` in later samplers or via `vars.get(...)` in later scripts — this is scripted chaining, the flexible cousin of extractors.

Common Mistakes

- **Choosing the wrong language** (e.g. beanshell) → slow and dated. Use **Groovy**.
- **vars.put with a non-String** → put values as `.toString()`; variables are strings.
- **Expecting `${var}` inside Groovy to auto-resolve** — inside a script, use `vars.get("var")`. `${var}` substitution happens in text fields, not reliably inside script bodies (it's evaluated once at compile if caching is on, which bites you). Prefer `vars.get()`.

Exercise 10

1. PreProcessor: generate `${email}` and a `${randomAge}`.
2. Use them in a POST body.
3. PostProcessor: parse the response and log the returned id.

4. JSR223 Assertion: fail the sample if the returned id is missing.
-

Section 11 — Dummy Sampler

What You'll Learn

Use the **Dummy Sampler** to fake a response, verify data flow, and debug extractions without hitting a real server.

Why This Matters

The Dummy Sampler is JMeter's equivalent of a Postman **mock** — but you'll mostly use it as a *debugging and teaching* tool. It lets you test extractors, assertions, and variable flow with a known, controlled "response," so you can isolate whether a problem is in your API or in your JMeter logic.

Install note: the Dummy Sampler comes from the **JMeter Plugins** ("jpgc - Dummy Sampler"). Install via the **Plugins Manager** (Options → Plugins Manager → Available Plugins → search "Dummy"). If Plugins Manager isn't present, drop jmeter-plugins-manager.jar into lib/ext/ and restart.

How To Do It

11.1 — Add a Dummy Sampler

Right-click Thread Group → **Add** → **Sampler** → **jp@gc - Dummy Sampler**. Fields:

- **Response Code:** 200
- **Response Message:** OK
- **Response Data:** paste any JSON you want to pretend the server returned.
- **Response Time / Latency:** simulate timings if you like.

11.2 — Debug an extractor against known data

Set Response Data to:

```
{ "id": 123, "name": "John", "status": "ACTIVE", "token": "abc123" }
```

Attach the multi-field **JSON Extractor** from Section 8 to this Dummy Sampler. Run. Add a Debug Sampler after it. You now know your extractor works — because the input was fixed and known. If the variables are wrong here, the bug is in your extractor, not the API.

11.3 — The workflow the workshop asks for

1. Hit a **real** API and extract values (`${id}`, `${token}`, ...) with Post-Processors.
2. Add a **Dummy Sampler** whose Response Data echoes those variables:
3. `{ "capturedId": "${id}", "capturedToken": "${token}", "capturedStatus": "${status}" }`
4. Run — the Dummy Sampler's response (visible in View Results Tree) shows the **resolved** values, proving what got passed through.
5. Attach a **JSR223 Assertion** to the Dummy Sampler to validate them:
6.

```
if (vars.get("token") == null || vars.get("token").isEmpty()) {  
    AssertionResult.setFailure(true)  AssertionResult.setFailureMessage("Token was  
    not captured from the API!")}
```

11.4 — Why this is so useful

- **Debugging:** confirm a variable holds what you think, without server noise.
- **Data-flow demos:** perfect for teaching — students see values flow between samplers.
- **Offline development:** build and test extractor/assertion logic when the real API is down.
- **Placeholder steps:** stub out a not-yet-built request in a chain so the rest of the flow runs.

Common Mistakes

- **Expecting Dummy Sampler to be built-in** — it's a plugin; install it first.
- **Confusing Dummy with Debug Sampler** — Debug Sampler dumps all variables; Dummy Sampler fakes a *response* you control (and can carry extractors/assertions).

Exercise 11

1. Real POST /auth → extract `${authToken}`.
2. Dummy Sampler with Response Data `{ "echoToken": "${authToken}" }`.
3. Confirm in View Results Tree that the Dummy response shows the real token string.
4. Add a JSR223 Assertion that fails if the token is blank.

Section 12 — Data-Driven Testing with CSV

What You'll Learn

Feed rows of test data from a CSV file into requests using **CSV Data Set Config**, and control how the data is shared, recycled, and terminated.

Why This Matters

This is the direct parallel to your Postman **Collection Runner + data file**. One request template + one CSV = many test cases and many simulated users, each with different credentials or payloads. It's how you avoid 20 duplicated requests.

The CSV file

Create users.csv (no quotes needed for simple values):

username,password,expectedCode

user1,password1,200

user2,password2,200

user3,wrongpass,403

Header row: JMeter can read the header names automatically, or you can list variable names manually. We'll use the manual approach for clarity.

12.1 — Add CSV Data Set Config

Right-click Thread Group → **Add** → **Config Element** → **CSV Data Set Config**. Configure:

Field	Value	Why
Filename	users.csv (relative to the .jmx, or absolute path)	The data source
Variable Names	username,password,expectedCode	Column → variable mapping
Ignore first line	True (if the CSV has a header row)	Skip the header
Delimiter	,	Column separator
Recycle on EOF	True/False	Loop back to row 1 when data runs out

Field	Value	Why
Stop thread on EOF	True/False	Stop a thread when data runs out
Sharing mode	All threads / Current thread group / Current thread	Who shares the row pointer

Now `${username}`, `${password}`, `${expectedCode}` are available in samplers.

12.2 — Use the variables in a request

POST /auth Body Data:

```
{ "username": "${username}", "password": "${password}" }
```

Response Assertion → Response Code → Equals → `${expectedCode}` (validate the *expected* code per row).

12.3 — Sharing mode explained (this trips people up)

- **All threads (default):** one shared pointer. Thread 1 gets row 1, thread 2 gets row 2, etc. Best for "each user logs in as a different person."
- **Current thread group:** shared only within this thread group.
- **Current thread:** each thread reads the file from the top independently. Best when every user should run *all* rows.

12.4 — Recycle on EOF & Stop thread on EOF

- **Recycle on EOF = True:** after the last row, wrap around to row 1. Use when you have fewer rows than iterations and want to keep going.
- **Recycle on EOF = False + Stop thread on EOF = True:** run each row exactly once, then stop. Use when "run every test case exactly once" (functional data-driven testing).
- **Recycle False + Stop False:** after EOF, variables get the literal `<EOF>` — usually a bug. Avoid.

Goal	Recycle	Stop
Run each row exactly once, then stop	False	True

Goal	Recycle Stop	
Loop rows forever under load	True	False

12.5 — Running multiple users with different data

Set Thread Group threads = 3, CSV sharing = **All threads**, Recycle = False, Stop = True.
Three users log in as user1/user2/user3 respectively — one row each, in parallel.

Common Mistakes

- **Wrong file path** → JMeter can't find the CSV; put it next to the .jmx and use a relative path.
- **Header row read as data** → set **Ignore first line = True** and don't also list the header as a value.
- **Variable name/column count mismatch.**
- **Trailing spaces / hidden BOM** in the CSV → weird values; save as plain UTF-8.
- **Recycle=False, Stop=False** → <EOF> values leak into requests.

Exercise 12

1. Create users.csv above.
2. Add CSV Data Set Config (Ignore first line = True, Recycle = False, Stop = True).
3. Thread Group = 3 threads, loop 1.
4. POST /auth per row; assert Response Code equals \${expectedCode}.
5. Confirm row 3 (wrong password) correctly expects and gets 403.

Section 13 — Dynamic Data

What You'll Learn

Generate values on the fly — random numbers/strings, timestamps, UUIDs, dynamic IDs, and dynamic JSON — using both built-in JMeter functions and Groovy.

Why This Matters

Static data causes collisions: run the same POST twice and you hit a duplicate-ID conflict. Dynamic data keeps every request unique, which is essential both for functional correctness and for realistic load (every user shouldn't send identical bytes).

13.1 — Built-in JMeter functions

JMeter functions use the `${__functionName(args)}` syntax and work directly in any field or body.

Function	Produces	Example
<code>\${__Random(1,1000)}</code>	Random integer in range	842
<code>\${__RandomString(8,abcdefABCDEF0123456789)}</code>	Random string from charset	a3F9bC1D
<code>\${__UUID()}</code>	A UUID v4	f47ac10b-58cc- ...
<code>\${__time()}</code>	Current epoch millis	1712345678901
<code>\${__time(yyyy-MM-dd)}</code>	Formatted date	2024-05-01
<code>\${__timeShift(yyyy-MM-dd,,P1D,,)}</code>	Date shifted by +1 day	tomorrow
<code>\${__counter(TRUE,)}</code>	Incrementing counter	1,2,3...
<code>\${__threadNum()}</code>	Current thread number	7
<code>` \${__RandomFromMultipleVars(a</code>	b	c)} `

13.2 — Dynamic JSON body with built-in functions

```
{  
  "firstname": "User_${__RandomString(5,abcdefghij)}",  
  "lastname": "Test",  
  "totalprice": ${__Random(50,500)},
```

```

"depositpaid": true,
"bookingdates": {
  "checkin": "${__time(yyyy-MM-dd)}",
  "checkout": "${__timeShift(yyyy-MM-dd,,P3D,,)}"
},
"reference": "${__UUID()}"
}

```

Every request now carries a unique name, price, dates, and reference.

13.3 — The Groovy approach (JSR223 PreProcessor)

More control, all in one place:

```

def rnd = new Random()

vars.put("randomPrice", (rnd.nextInt(450) + 50).toString())
vars.put("uuid", UUID.randomUUID().toString())
vars.put("checkin", new Date().format("yyyy-MM-dd"))
vars.put("checkout", (new Date() + 3).format("yyyy-MM-dd"))
vars.put("firstname", "User_" + rnd.nextInt(99999))

```

Then reference `${randomPrice}`, `${uuid}`, `${checkin}`, `${checkout}`, `${firstname}` in the body.

13.4 — Built-in functions vs Groovy — which to use?

Use built-in functions when... Use Groovy when...

Simple one-off value inline	Multiple related values
Quick and readable	Conditional/computed logic
No extra element needed	You need loops, parsing, or math

Common Mistakes

- **Quoting a numeric function** — `"totalprice": "${__Random(1,9)}"` sends a string; drop the quotes for numbers.

- **`${__RandomString}` charset omitted** → second arg is the alphabet; if blank you get odd output.
- **Reusing `${__UUID()}` expecting the same value twice** — each evaluation is fresh. Capture it into a variable first if you need to reuse it.

Exercise 13

1. Build a POST /booking whose name, price, and dates are all dynamic (try the built-in version, then the Groovy version).
2. Run with 5 threads and confirm in View Results Tree that every request body differs.

Section 14 — Assertions

What You'll Learn

The main assertion types, what each checks, and when to reach for which.

Why This Matters

Assertions are your `pm.test(...)`. Without them, a load test measures speed but never checks *correctness* — you could be blazingly fast at returning error pages. Good tests assert both.

The assertion toolbox

Assertion	Checks	Postman analog
Response Assertion	Status code, body text, headers (Contains/Matches/Equals)	<code>pm.response.to.have.status,</code> <code>.include</code>
JSON Assertion	A JSON Path exists / equals a value	<code>pm.expect(body.x).to.eql(...)</code>
JSON JMESPath Assertion	JMESPath query result	advanced JSON checks
Duration Assertion	Response time under a threshold	<code>responseTime</code> check
Size Assertion	Response size within bounds	(no direct analog)

Assertion	Checks	Postman analog
XPath2 Assertion	XML/HTML structure	XML checks
JSR223 Assertion	Any custom Groovy logic	pm.test with arbitrary JS
MD5Hex Assertion	Response bytes match a hash	integrity check

14.1 — Response Assertion (the workhorse)

Check status:

- Field: **Response Code**, Rule: **Equals**, Pattern: 200.

Check body contains text:

- Field: **Text Response**, Rule: **Contains**, Pattern: "bookingid".

Check absence (negative test):

- Field: **Text Response**, Rule: **Contains**, Pattern: error, tick **Not**.

14.2 — JSON Assertion

- Assert JSON Path exists: \$.booking.firstname.
- Additionally assert value: Expected Jim.

14.3 — JSON JMESPath Assertion

For queries JSON Path can't do — e.g. assert a value inside a filtered array:

- JMESPath: booking.bookingdates.checkin, expected 2024-01-01.

14.4 — Duration Assertion

- Duration in milliseconds: 1500. Sample fails if slower. Great as a per-request SLA.

14.5 — Size Assertion

- Response Size **Greater than** 0 bytes — catches empty responses.
- Or **Less than** some max to catch bloated responses.

14.6 — JSR223 Assertion (custom)

```

import groovy.json.JsonSlurper

def json = new JsonSlurper().parseText(prev.getResponseDataAsString())

if (json.booking?.totalprice == null) {

    AssertionResult.setFailure(true)

    AssertionResult.setFailureMessage("Missing totalprice in booking")

} else if (json.booking.totalprice <= 0) {

    AssertionResult.setFailure(true)

    AssertionResult.setFailureMessage("totalprice must be positive, got " +
    json.booking.totalprice)

}

```

14.7 — Scope & combining

- Multiple assertions on one sampler → **all** must pass or the sample fails.
- An assertion under the Thread Group applies to **every** sampler in scope (e.g. a global "must be 2xx" check).
- View pass/fail in **View Results Tree** (red/green) or the **Assertion Results** listener.

Common Mistakes

- **Only asserting status code** — the recurring lesson: check the body too.
- **Matches vs Contains** — Matches requires the whole response to match the regex; use Contains for substrings.
- **Assertion scoped too broadly** — a Thread-Group-level assertion may fail on samplers it wasn't meant for.

Exercise 14

For GET /booking/1 add, all at once: a Response Assertion (200), a JSON Assertion (\$.firstname exists), a Duration Assertion (2000ms), and a Size Assertion (>0). Break each one deliberately to see it fail red.

Section 15 — Controllers

What You'll Learn

Logic Controllers that decide **which** samplers run and **how often** — the flow-control layer of a test plan.

Why This Matters

Real workflows aren't a flat list. You log in **once**, then loop through actions, sometimes conditionally. Controllers let you express "do this only if...", "repeat this block", "group these as one transaction." They're your if, for, and folders combined.

The controllers you'll actually use

Simple Controller — *What*: a plain container/folder, no logic. *Why*: organize samplers into named groups. *Example*: wrap "Booking flow" samplers so the tree is readable.  **From Postman**: a folder.

Transaction Controller — *What*: groups child samplers and reports their **combined** time as one entry. *Why*: measure an end-to-end business transaction (e.g. "Checkout" = 3 requests) as a single number. Tick "Generate parent sample." *Example*: time the whole login+create+get chain as "CreateBooking".

Loop Controller — *What*: repeats its children N times. *Why*: repeat a block without adding users. *Example*: Loop 5 → post 5 bookings per user.

Once Only Controller — *What*: runs its children only on the **first** iteration of each thread. *Why*: log in once, not on every loop. *Example*: put POST /auth inside it so the token is fetched a single time.  **From Postman**: a setup step you didn't want repeated.

If Controller — *What*: runs children only if a condition is true. *Why*: branch on data or previous results. *Example*: condition `${__jexl3("${statusCode}" == "200")}` → only proceed if login succeeded.

Prefer the JEXL/Groovy form and tick "**Interpret Condition as Variable Expression**" appropriately; avoid the legacy JavaScript evaluation for performance.

While Controller — *What*: loops while a condition holds. *Why*: poll until something happens. *Example*: keep calling GET /job/\${id} while `${status} != DONE`.

ForEach Controller — *What*: iterates over a set of variables like item_1, item_2, ... produced by an extractor. *Why*: process every extracted value. *Example*: an extractor pulled all booking IDs into bookingId_1..N; ForEach fetches each one.

Throughput Controller — *What*: runs children only a **percentage** of the time (or a set number of executions). *Why*: mix traffic realistically — e.g. 70% browse, 30% checkout. *Example*: set "Percent Executions = 30" on the checkout block.

Runtime Controller — *What:* runs children for a fixed number of seconds. *Why:* time-box a block.

15.1 — A realistic combined example

Thread Group (20 users, loop 10)

```
|— Once Only Controller
|  |— POST /auth → extract ${authToken}
|  |— Transaction Controller "Create+Read Booking"
|    |— POST /booking → extract ${bookingId}
|    |— GET /booking/${bookingId}
|— Throughput Controller (Percent = 30)
    |— DELETE /booking/${bookingId} (only ~30% of iterations delete)
```

This says: each user logs in once, then on every loop creates+reads a booking (timed as one transaction), and 30% of the time also deletes it.

Common Mistakes

- **Login on every loop** — forgetting the Once Only Controller re-authenticates each iteration, skewing results.
- **ForEach variable prefix mismatch** — the input variable names must match the extractor's output prefix exactly.
- **If Controller evaluated every time is expensive** — keep conditions light; prefer JEXL3/Groovy over the deprecated JS engine.

Exercise 15

Build the combined example above. Verify in the Summary Report that "Create+Read Booking" appears as **one** row (Transaction Controller) and that DELETE runs far fewer times than the create step (Throughput Controller).

Section 16 — Timers

What You'll Learn

The main timers and why pauses make a load test realistic.

Why This Matters

Real users pause — they read, they think, they type. Fire requests with zero gap and you create an artificial hammer that no real traffic resembles, often overstating load. Timers add human-like think-time and let you control throughput. (Postman had no equivalent — this is pure load-testing craft.)

16.1 — Constant Timer

What: a fixed pause before each sampler in scope. *Example:* Thread Delay 1000 ms → every request waits 1s. *Use:* simple, uniform think-time.

16.2 — Uniform Random Timer

What: random pause = a constant offset + a random component evenly spread. *Fields:* "Constant Delay Offset" + "Random Delay Maximum." *Example:* offset 1000, random 2000 → pauses between 1.0s and 3.0s, uniformly. *Use:* realistic jitter so users don't move in lockstep.

16.3 — Gaussian Random Timer

What: like Uniform but the randomness follows a bell curve (most pauses near the mean). *Example:* offset 1000, deviation 500 → most pauses ~1s, tapering off. *Use:* the most human-like distribution of think-time.

16.4 — Constant Throughput Timer

What: paces threads to hit a **target throughput** (samples per minute), adding delay as needed. *Example:* target 600 samples/minute = 10 requests/second total. *Use:* when the requirement is "sustain X requests/sec" rather than "N users." Choose the calculation basis (all threads, this thread only, etc.) carefully.

16.5 — Precise Throughput Timer

What: a newer, more accurate throughput shaper (samples per time unit with a defined test duration). *Use:* modern alternative to Constant Throughput Timer for precise RPS targets.

Scope reminder

A timer applies to **all samplers in its scope** and executes **before** each sampler. Put it under the Thread Group for global think-time, or under one sampler to delay just that request.

Choosing a timer

Need	Timer
Fixed pause everywhere	Constant Timer
Realistic even jitter	Uniform Random Timer
Human bell-curve think-time	Gaussian Random Timer
Hit an exact requests/sec target	Constant / Precise Throughput Timer

Common Mistakes

- **No timers at all** → unrealistic hammering and inflated load.
- **A timer under the Thread Group when you meant one sampler** → everything slows down.
- **Throughput Timer basis misunderstood** — "all active threads" vs "this thread only" changes the math a lot.

Exercise 16

Take your booking chain. Add a Uniform Random Timer (offset 500, random 1500). Run 10 users and compare the throughput to a run with no timer — note how think-time lowers RPS but models reality.

Section 17 — Correlation

What You'll Learn

The concept of **correlation** — capturing server-generated dynamic values and replaying them — and a complete worked example.

Why This Matters

Correlation is the single most important skill in real-world load testing. Servers hand out dynamic values (session tokens, CSRF tokens, generated IDs) that you cannot hardcode — they're different every run and every user. If you don't correlate them, your test fails with 401/403/404 the moment it leaves the login step. (In Section 9 you did chaining; correlation is the same mechanism applied specifically to *dynamic server-generated* values under load.)

17.1 — What correlation means

Correlation = extract a dynamic value from response N, then inject it into request N+1.

"Recording" a test with hardcoded tokens will pass once and fail forever after, because the token changes. Correlation replaces the recorded static value with a freshly-extracted one.

17.2 — Why dynamic values need it

Value	Why it's dynamic	If you hardcode it
Auth/session token	New per login	401 Unauthorized
CSRF token	New per page load	403 Forbidden
Generated resource ID	Assigned by server on create	404 Not Found
ViewState / nonce	New per request	Request rejected

17.3 — The correlation toolkit

1. **Extract** with a Post-Processor: JSON Extractor, Regex Extractor, or Boundary Extractor.
2. **Store** in a JMeter variable (\${token}, \${id}).
3. **Reuse** in the next request's header/path/body.

17.4 — Complete correlation example (Restful-Booker)

Step 1 POST /auth

Body: {"username":"admin","password":"password123"}

Post-Processor: JSON Extractor \$.token → \${authToken}

Assertion: 200

Step 2 POST /booking (dynamic body from Section 13)

Post-Processor: JSON Extractor \$.bookingid → \${bookingId}

Assertion: 200, JSON \$.booking.firstname exists

Step 3 GET /booking/\${bookingId}

Assertion: 200; JSON \$.firstname matches created value

Step 4 PUT /booking/\${bookingId}

Header Manager: Cookie: token=\${authToken} ← correlated token

Body: updated fields

Assertion: 200

Step 5 DELETE /booking/\${bookingId}

Header Manager: Cookie: token=\${authToken} ← correlated token

Assertion: 201

Two correlated values thread through the flow: \${authToken} (from step 1) and \${bookingId} (from step 2). Every dynamic value the server produced is captured and replayed.

17.5 — Correlation with the Regex Extractor (non-JSON)

If a token appears in HTML or a header, JSON Path won't help. Use a Regex Extractor:

- **Reference Name:** csrfToken
- **Regular Expression:** name="csrf" value="([^\"]+)"
- **Template:** \$1\$
- **Match No.:** 1

Debugging correlation

- Add a **Debug Sampler** after each extractor to confirm the variable populated.
- Check View Results Tree → the **Request** tab of the *next* sampler to confirm the value was actually substituted (not left as \${authToken}).

Common Mistakes

- **Recording a script and not correlating** → passes once, then 401/403 forever.
- **Right value, wrong scope** → token stored as a property (global) but needed per-user as a variable, causing users to clobber each other.
- **Extractor after the sampler that needs the value** → order matters.

Exercise 17

Take the correlation example and deliberately hardcode `{authToken}` to a stale string in step 4. Watch it 403. Restore the correlation and watch it pass. That contrast is the whole lesson.

Section 18 — Load Testing Concepts

What You'll Learn

The vocabulary and types of performance testing, and the metrics you'll judge results by. This is the dimension Postman never had.

Why This Matters

Up to now you've done **functional** API testing — does the endpoint behave correctly? Load testing asks a different question — does it behave correctly, and *fast enough*, when many users hit it at once? Different goal, same tool. Getting the terminology right lets you design the right test for the right question.

18.1 — Types of performance testing

Type	Question it answers	Typical shape
Load testing	Does it meet SLAs at expected traffic?	Ramp to expected peak, hold
Stress testing	Where does it break?	Increase load past the limit
Spike testing	Can it survive a sudden surge?	Jump from low to very high instantly
Endurance / Soak testing	Does it degrade over time (memory leaks)?	Moderate load for hours
Volume testing	How does it handle large data sets?	Big payloads / many records
Scalability testing	Does adding capacity help proportionally?	Increase load and resources together

Functional vs performance — keep them separate in your head. A functional test asserts *correctness* with 1 user. A performance test measures *behavior under concurrency*. You often reuse the same samplers, but the intent and the reporting differ.

18.2 — The metrics that matter

Metric	Meaning	What "good" looks like
Concurrent users	Threads active at once	Matches your target scenario
Throughput (RPS)	Requests completed per second	Meets/exceeds expected traffic
Response time	Full round-trip per request	Under your SLA (e.g. <500ms)
Latency	Time to first byte	Low and stable
Percentiles (p90/p95/p99)	"X% were faster than..."	p95 under SLA; watch p99 for outliers
Error rate (%)	Failed requests	As close to 0% as possible
Ramp-up / Ramp-down	How load rises/falls	Gradual, controlled
Std deviation	Consistency of response times	Low = predictable

18.3 — Why percentiles beat averages

An **average** hides pain. If 95 requests take 100ms and 5 take 10,000ms, the average looks okay-ish but 1 in 20 users had a terrible experience. **p95 = 10,000ms** exposes that. Always report p90/p95/p99 alongside the average.

18.4 — Reading the shape of a load test

As users increase:

- **Healthy zone:** throughput rises, response time stays flat.
- **Saturation point:** throughput plateaus, response time starts climbing.
- **Overload:** throughput drops, response time spikes, errors appear.

The saturation point is your capacity limit — the number you report.

Common Mistakes

- **Reporting only the average.**
- **Confusing latency and response time** — latency is to first byte; response time is the full response.
- **No ramp-down / abrupt stop** — can distort the tail metrics.

- **Running a "load test" with no assertions** — fast garbage is still garbage.

Exercise 18

Match each scenario to a test type: (a) "Black Friday sale traffic at midnight," (b) "does the app leak memory over an 8-hour shift," (c) "find the max users before it 500s." (Answers: a=spike, b=endurance/soak, c=stress.)

Section 19 — Load Test Example

What You'll Learn

Build one complete beginner load test, understand exactly what JMeter does with it, then scale it up in stages and interpret each stage.

Why This Matters

Concepts stick when you watch numbers move. Here you turn a functional test into a load test and see saturation happen with your own eyes.

19.1 — The baseline load test

Thread Group settings:

Number of Threads (users): 100

Ramp-up period: 60 seconds

Loop Count: 10

What JMeter does, precisely:

- Starts **100 virtual users**, adding one roughly every 0.6 seconds (100 users ÷ 60s).
- By second 60, all 100 are active.
- Each user runs the samplers **10 times** (Loop = 10).
- Total requests per sampler = $100 \times 10 = \mathbf{1,000}$.
- If there are 3 samplers in the flow, that's **3,000 requests** total.

What to watch (in Summary/Aggregate Report):

- **Throughput** — settles to the server's processing rate.
- **Average & p95 response time** — should stay under your SLA.

- **Error %** — ideally 0.

Use a public API like Restful-Booker *gently* for learning; never point heavy load at a service you don't own. For real load practice, run the target API locally.

19.2 — Scale it up in stages

Run the same plan at increasing user counts, recording results each time:

10 users

↓

25 users

↓

50 users

↓

100 users

↓

200 users

Make this easy by parameterizing: Number of Threads = $\$_{__P}(\text{users}, 10)$, Ramp-up = $\$_{__P}(\text{rampup}, 30)$, then:

```
jmeter -n -t load.jmx -l results-10.jtl -Jusers=10 -Jrampup=10
```

```
jmeter -n -t load.jmx -l results-25.jtl -Jusers=25 -Jrampup=15
```

```
jmeter -n -t load.jmx -l results-50.jtl -Jusers=50 -Jrampup=30
```

```
jmeter -n -t load.jmx -l results-100.jtl -Jusers=100 -Jrampup=60
```

```
jmeter -n -t load.jmx -l results-200.jtl -Jusers=200 -Jrampup=120
```

19.3 — What students should observe at each stage

Stage	Expected observation
-------	----------------------

10 users	Fast, flat response times. Baseline throughput. 0% errors.
-----------------	--

25 users	Throughput rises roughly proportionally. Response times still flat.
-----------------	---

50 users	Throughput keeps rising; response time may tick up slightly.
-----------------	--

Stage	Expected observation
100 users	Approaching saturation — throughput growth slows, p95 climbs.
200 users	Likely past saturation — throughput flattens or drops, response times spike, first errors appear.

The stage where **throughput stops rising but response time keeps climbing** is your **saturation point** — the headline capacity number.

19.4 — Turning the functional chain into a load test

You already built the booking chain (Section 9/17). To load-test it:

1. Keep the samplers and extractors as-is.
2. Increase threads and loops.
3. Add a **Uniform Random Timer** for think-time.
4. Wrap login in a **Once Only Controller** so you don't re-auth every loop.
5. **Remove/disable heavy GUI listeners** (see Section 22) and run **Non-GUI**.

Common Mistakes

- **All users starting at once (ramp-up 0)** → self-inflicted spike that crashes your load generator, not the server.
- **Running the scale-up in GUI mode** → the tool becomes the bottleneck.
- **Not saving each stage's .jtl** → you lose the comparison.

Exercise 19

Run the 5-stage scale-up against a **local** test API. Plot users (x) vs throughput and p95 (y). Circle the saturation point.

Section 20 — Results & Reporting

What You'll Learn

The main listeners, the metrics they show, and — critically — which listeners to avoid during real load tests.

Why This Matters

Results are useless if you can't read them, and misleading if you collect them the wrong way. Heavy listeners can slow JMeter enough to corrupt the very numbers you're trying to measure.

20.1 — The key listeners

Listener	Shows	Best for
View Results Tree	Every request/response in full	Debugging only (1–2 users)
Summary Report	Aggregate stats per sampler	Quick overview
Aggregate Report	Adds median & percentiles	Analyzing a load run
Response Time Graph	Response time over the run	Trend visualization
Aggregate Graph	Bar charts of aggregate stats	Presentations
Backend Listener	Streams metrics to InfluxDB/Graphite	Live Grafana dashboards
Simple Data Writer	Writes raw results to .jtl with minimal overhead	Non-GUI runs

20.2 — Metrics in the Aggregate Report (read these columns)

Column	Meaning
# Samples	Requests sent
Average	Mean response time
Median	50th percentile (half were faster)
90% Line (p90)	90% were faster than this
95% Line (p95)	95% were faster than this
99% Line (p99)	99% were faster than this — outlier watch
Min / Max	Fastest / slowest

Column	Meaning
Error %	Failure rate
Throughput	Requests/sec (or /min)

Received/Sent KB/sec Bandwidth

Report p90/p95/p99, not just Average — the whole point of Section 18.3.

20.3 — The Backend Listener (live dashboards)

For serious load tests, add a **Backend Listener** with the InfluxDB implementation to stream metrics into InfluxDB, then visualize live in **Grafana**. This gives real-time graphs during the run without the overhead of GUI listeners.

20.4 — Which listeners NOT to use during load tests

⚠ Do not use View Results Tree, Aggregate Graph, or other GUI-heavy, per-sample listeners during a real load test. They keep **every sample in memory** and render live, which:

- consumes huge RAM (out-of-memory on big runs),
- burns CPU on rendering,
- **slows JMeter itself**, inflating your response-time numbers.

Instead, for load runs:

- Run **Non-GUI** and write results with -l results.jtl (a lightweight Simple Data Writer under the hood).
- Generate the **HTML Dashboard** afterward (Section 21) for all the graphs you need.
- Use View Results Tree **only** while building/debugging with a couple of users. If you keep one in the plan, set it to **log errors only** or disable it before the load run.

Common Mistakes

- **Leaving View Results Tree enabled** during a 500-user run → skewed results and possible OOM.
- **Judging by Average alone.**
- **Not writing a .jtl** → nothing to analyze or turn into a dashboard.

Exercise 20

1. Run a small load test in GUI with View Results Tree enabled; note the throughput.
 2. Disable it, run again; compare. Observe how much the listener itself cost.
 3. Open the Aggregate Report and read off p90/p95/p99.
-

Section 21 — HTML Dashboard Report

What You'll Learn

Generate JMeter's professional HTML dashboard from a test run, understand each command flag, and interpret the graphs.

Why This Matters

This is your **Newman HTML report** equivalent — a polished, shareable artifact you hand to developers and managers. It turns a raw .jtl into charts, percentile tables, and over-time graphs.

21.1 — Generate the dashboard in one command

```
jmeter -n -t test.jmx -l results.jtl -e -o report
```

What each part means:

Part	Meaning
-n	Non-GUI mode (required for real runs)
-t test.jmx	The test plan to execute
-l results.jtl	Write raw results here (created by the run)
-e	After the run, generate the report dashboard
-o report	Output the HTML dashboard into the report/ folder
• The .jtl appears wherever you pointed -l (here, current folder). • The HTML report lands in the report/ folder (-o). • The -o folder must be empty or not exist — JMeter refuses to overwrite an existing report.	

21.2 — Generate a report from an existing .jtl

Already have a .jtl from an earlier run? Build the dashboard without re-running:

```
jmeter -g results.jtl -o report
```

21.3 — Open the report

Open report/index.html in any browser (double-click, or open report/index.html / start report\index.html).

21.4 — What the dashboard contains

- **APDEX table** — an application performance index (satisfied/tolerating/frustrated) per request.
- **Requests Summary** — pass/fail donut.
- **Statistics table** — per-request samples, average, min, max, **p90/p95/p99**, throughput, error %.
- **Errors table** — breakdown of failures by type.
- **Over-Time graphs** — response times, throughput, active threads, latency across the run.
- **Response-time percentiles & distribution** charts.

21.5 — Interpreting the graphs

- **Response Times Over Time** climbing while **Active Threads** rises → approaching saturation.
- **Throughput Over Time** flattening while threads still increase → capacity limit reached.
- **Percentiles chart** — a long tail (p99 >> p95) signals inconsistent outliers worth investigating.

21.6 — Making the report more informative/professional

- Set meaningful **sampler names** — they become the report labels. Name them by business action ("Login", "Create Booking"), not "HTTP Request".
- Use **Transaction Controllers** so end-to-end flows appear as single rows.

- Configure **APDEX thresholds** in user.properties (jmeter.reportgenerator.apdex_satisfied_threshold=500, ...tolerated_threshold=1500) to match your SLA.
- Split traffic types (browse vs checkout) into separate transactions so the report shows each independently.
- Keep the .jtl format as **CSV** for smaller files and faster report generation.

Common Mistakes

- **Reusing a non-empty -o folder** → error. Use a fresh, dated folder: -o report-2024-05-01-100u.
- **Generating a dashboard from a GUI run with heavy listeners** → the numbers are already tainted; always run Non-GUI.
- **Unnamed samplers** → the report is a wall of "HTTP Request" rows.

Exercise 21

1. Run any load plan with -n -t ... -l results.jtl -e -o report.
2. Open report/index.html.
3. Find the p95 for your main request and the APDEX score.
4. Re-generate a report from the same .jtl using -g into a new folder.

Section 22 — GUI vs Non-GUI Best Practices

What You'll Learn

When to use each mode and why production load tests must run headless.

Why This Matters

This is the rule that separates people who get trustworthy numbers from people who don't. It's worth stating flatly and repeating.

22.1 — The two phases

Development / debugging phase → GUI mode

- Build the tree, wire extractors, verify assertions.
- Use **View Results Tree** and **Debug Sampler** with **1–2 users**.

- Iterate quickly with instant visual feedback.

Actual load testing → Non-GUI mode

`jmeter -n -t test.jmx -l results.jtl -e -o report`

- Headless, low overhead, accurate.
- Handles hundreds/thousands of users.

22.2 — Why GUI listeners hurt performance

Live GUI listeners (especially **View Results Tree** and graph listeners):

- store **every sample in memory** → memory blows up on large runs,
- **render continuously** → CPU spent drawing instead of generating load,
- make **JMeter itself the bottleneck** → response times get inflated by the tool, not the server.

The result: your "server is slow" finding is actually "my listener is slow." Non-GUI mode removes that variable entirely.

22.3 — The professional checklist before a load run

- [] Test verified in GUI with 1–2 users.
- [] Heavy listeners removed or disabled (keep only lightweight result writing).
- [] Parameterized with `-J` for users/ramp/host.
- [] Adequate heap set (HEAP env var, e.g. `-Xmx4g`) for the load generator.
- [] Running **Non-GUI**: `jmeter -n -t ... -l ... -e -o ....`
- [] Results written to `.jtl`; dashboard generated after.

Common Mistakes

- **Load testing in GUI mode** — the cardinal sin. JMeter literally warns you against it on startup.
- **Forgetting to disable View Results Tree** before the big run.
- **Under-sized heap** on the load generator → OOM mid-test.

Exercise 22

Run the same 50-user test twice: once in GUI with View Results Tree on, once Non-GUI. Compare throughput and p95. Write one sentence explaining the difference.

Section 23 — Debugging & Troubleshooting

What You'll Learn

The common failures you'll hit and how to diagnose each fast.

Why This Matters

Most workshop time gets lost to small, fixable issues. Knowing the usual suspects turns a 20-minute stall into a 20-second fix.

23.1 — First-line debugging tools

- **View Results Tree** → click the sampler → read **Sampler result / Request / Response data**.
- **Debug Sampler** → dumps all current variables; the fastest way to see if `$(x)` is set.
- **jmeter.log** (bottom of GUI, or the file in bin/) → stack traces for script errors.
- **View Results Tree** → **Request tab** → confirms whether `$(var)` was substituted or sent literally.

23.2 — Symptom → cause → fix

Symptom	Likely cause	Fix
<code>\$(var)</code> appears literally in the request	Variable never set / typo / wrong scope	Add Debug Sampler; check extractor ran and name matches
Extractor returns default / empty	Wrong JSON Path/regex, or Match No. = 0	Test the path; set Match No. = 1; check it's on the right sampler
HTTP 400 Bad Request	Malformed JSON, wrong Content-Type, JSON in Parameters tab	Validate JSON; ensure Body Data + Content-Type: application/json

Symptom	Likely cause	Fix
HTTP 401 Unauthorized	Missing/expired/empty token	Confirm <code>{authToken}</code> populated; check Authorization header
HTTP 403 Forbidden	Token not sent where the API expects (e.g. Cookie vs Bearer)	Match the API's auth scheme exactly
HTTP 404 Not Found	Empty path variable (<code>/booking/</code>), wrong URL	Ensure the ID extracted before use; verify path
HTTP 415 Unsupported Media Type	Missing/wrong Content-Type	Add Content-Type: <code>application/json</code>
HTTP 500 Server Error	Bad payload or a genuine server bug	Inspect request body; reproduce in isolation
Connection refused / timeout	Server down, wrong host/port, firewall	Check host/port; increase timeouts; verify server up
Non-HTTP response / SSL errors	Protocol mismatch, cert issues	Correct http/https; adjust SSL settings if needed
CSV values wrong / <EOF>	Bad path, header read as data, <code>Recycle=False+Stop=False</code>	Fix path; Ignore first line = True; set Recycle/Stop correctly
JSR223 error in log	Groovy syntax, non-String vars.put, wrong object	Read the stack trace; <code>.toString()</code> values; verify prev/vars/props usage
Everything slow / OOM	Heavy GUI listeners, small heap	Run Non-GUI; remove listeners; raise <code>-Xmx</code>

23.3 — A repeatable debugging method

1. **Isolate** — disable other samplers; run just the failing one.
2. **Inspect the Request tab** — is the URL/body/headers what you intended?
3. **Inspect the Response** — what did the server actually say?

4. **Add a Debug Sampler** — are the variables set?
5. **Check `jmeter.log`** — any script/parse errors?
6. **Fix one thing, re-run.**

Common Mistakes

- **Changing five things then re-running** — you won't know which fixed (or broke) it. Change one at a time.
- **Ignoring `jmeter.log`** — the answer is usually right there.

Exercise 23

Deliberately introduce three bugs into your booking chain — a malformed JSON body, a wrong JSON Path in an extractor, and a missing auth header — then use the table and method above to diagnose and fix each.

Section 24 — Complete Hands-On Mini Project

What You'll Learn

Combine everything into one realistic, data-driven, chained API test — then convert that same test into a load test.

Why This Matters

This is your capstone, the JMeter equivalent of your Postman bug-hunting finale. It proves you can assemble config, data, chaining, extraction, assertions, and reporting into a single professional test plan — and then flip it into performance mode.

The project flow

Read test data from CSV

↓

Login API → extract `${authToken}`

↓

Create resource → extract `${bookingId}`

↓

Get resource → validate response

↓

Update resource → uses \${authToken}

↓

Delete resource → uses \${authToken}

Target API: **Restful-Booker** (<https://restful-booker.herokuapp.com>).

24.1 — Build the functional version (step by step)

Test Plan booking-mini-project.jmx

- **User Defined Variables:** baseUrl=restful-booker.herokuapp.com, protocol=https.

Config (Thread Group level)

- **HTTP Request Defaults:** Protocol \${protocol}, Server \${baseUrl}.
- **HTTP Header Manager:** Content-Type: application/json, Accept: application/json.
- **CSV Data Set Config:** file bookings.csv, variables firstname,lastname,price, Ignore first line = True, Recycle = False, Stop thread on EOF = True.

bookings.csv:

firstname,lastname,price

Jim,Brown,111

Sally,Green,222

Bob,White,333

Thread Group (functional: 3 threads, 1 loop, ramp-up 3)

1. Once Only Controller → POST /auth

- Body: {"username":"admin","password":"password123"}
- JSON Extractor: \$.token → authToken
- Response Assertion: code = 200

2. Transaction Controller "Booking Lifecycle"

a. POST /booking

- Body (data-driven + dynamic dates):

- { "firstname": "\${firstname}", "lastname": "\${lastname}", "totalprice": \${price}, "depositpaid": true, "bookingdates": { "checkin": "\${__time(yyyy-MM-dd)}", "checkout": "\${__timeShift(yyyy-MM-dd,,P2D,,)}" }, "additionalneeds": "Breakfast" }
- JSON Extractor: \$.bookingid → bookingid
- Assertions: code = 200; JSON \$.booking.firstname = \${firstname}

b. GET /booking/\${bookingid}

- Assertions: code = 200; JSON \$.totalprice = \${price}
- (Add a **Debug Sampler** here while building.)

c. PUT /booking/\${bookingid}

- Header Manager (this sampler): Cookie: token=\${authToken}
- Body: same shape, firstname = \${firstname}Updated
- Assertions: code = 200; JSON \$.firstname = \${firstname}Updated

d. DELETE /booking/\${bookingid}

- Header Manager (this sampler): Cookie: token=\${authToken}
- Assertion: code = 201

3. **Listeners:** View Results Tree + Summary Report (for building only).

Run it with 3 threads. Each thread takes a different CSV row and runs the full lifecycle. Confirm all green.

24.2 — Convert the SAME plan into a load test

Change nothing about the logic — only the load profile and hygiene:

1. **Thread Group:** threads \${__P(users,100)}, ramp-up \${__P(rampup,60)}, loop \${__P(loops,10)} (or use the Scheduler for a duration).
2. **CSV Data Set Config:** set **Recycle on EOF = True** (so 100 users can keep reusing the 3 rows) and **Stop thread on EOF = False**.
3. **Add a Uniform Random Timer** (offset 500, random 1500) for think-time.
4. **Remove/disable** View Results Tree and any graph listeners.
5. **Run Non-GUI with a dashboard:**

6. `jmeter -n -t booking-mini-project.jmx -l results.jtl -e -o report \ -Jusers=100 -Jrampup=60 -Jloops=10`
7. **Open report/index.html** and read throughput, p95, and error %.

24.3 — Scale it up

Re-run at 10 → 25 → 50 → 100 → 200 users (as in Section 19), each to its own .jtl/report, and identify the saturation point.

24.4 — What you've demonstrated

- Config elements (Defaults, Header Manager, UDV) ✓
- Data-driven testing (CSV) ✓
- Correlation & chaining (token + booking id) ✓
- Multi-field extraction & assertions ✓
- Dynamic data (dates) ✓
- Controllers (Once Only, Transaction) ✓
- Timers ✓
- Non-GUI execution + HTML dashboard ✓
- Functional → load conversion ✓

That's the full arc of the workshop in one artifact.

Final Challenge

Add a **negative** row to the CSV (wrong price type or empty name), give the create step an assertion that expects the appropriate error code for that row (data-driven expected codes, Section 12), and prove your suite catches both the happy path and the failure path — the same "find how it breaks" mindset from your Postman bug-hunt.

Quick Reference

JMeter Component Cheat Sheet

I want to...	Use this component
Set the host once	HTTP Request Defaults
Set shared headers	HTTP Header Manager
Define reusable variables	User Defined Variables
Feed data rows	CSV Data Set Config
Send a request	HTTP Request (Sampler)
Extract from JSON	JSON Extractor / JSON JMESPath Extractor
Extract from text/HTML	Regular Expression / Boundary Extractor
Prepare data before a request	JSR223 PreProcessor
Process/extract after a request	JSR223 PostProcessor
Validate status/body/headers	Response Assertion
Validate a JSON value	JSON Assertion
Validate response time	Duration Assertion
Custom validation	JSR223 Assertion
Run something once per user	Once Only Controller
Branch on a condition	If Controller
Group requests as one timed unit	Transaction Controller
Add think-time	Constant / Uniform / Gaussian Timer
Hit a target RPS	Constant / Precise Throughput Timer
Debug variables	Debug Sampler
Fake a response	Dummy Sampler (plugin)
See each request/response	View Results Tree (debug only)
See aggregate stats	Aggregate Report

Command-Line Cheat Sheet

Command	Purpose
jmeter	Open GUI
jmeter -t test.jmx	Open GUI with a plan loaded
jmeter -n -t test.jmx -l results.jtl	Run headless, write results
jmeter -n -t test.jmx -l results.jtl -e -o report	Run headless + HTML dashboard
jmeter -g results.jtl -o report	Build dashboard from an existing .jtl
jmeter -n -t test.jmx -l r.jtl -Jusers=100 -Jrampup=60	Pass properties in
jmeter -n -t test.jmx -q qa.properties -l r.jtl	Load an env properties file

\${...} Syntax Cheat Sheet

Expression	Meaning
`\${myVar}`	A JMeter variable
`\$__P(prop,default)`	A property (from -J/file) with fallback
`\$__Random(1,100)`	Random integer
`\$__RandomString(8,abc123)`	Random string from charset
`\$__UUID()`	A UUID
`\$__time(yyyy-MM-dd)`	Current formatted date
`\$__timeShift(yyyy-MM-dd,,P1D,,)`	Shifted date (+1 day)
`\$__counter(TRUE,)`	Per-thread incrementing counter
`\$__threadNum()`	Current thread number

Groovy Built-in Objects Cheat Sheet

```
vars.get("name"); vars.put("name","value"); // thread variables  
props.get("name"); props.put("name","value"); // global properties  
prev.getResponseCode(); // "200"
```

```

prev.getResponseDataAsString();          // response body
prev.getResponseHeaders();               // headers
ctx.getThreadNum();                      // thread index
log.info("msg"); log.warn("msg"); log.error("m");// logging
AssertionResult.setFailure(true);        // (JSR223 Assertion only)
AssertionResult.setFailureMessage("why");

```

HTTP Status Codes (same as your Postman sheet)

Code	Meaning	Common cause in JMeter
200	OK	Success
201	Created	Successful POST (varies by API)
400	Bad Request	Malformed JSON / wrong body location
401	Unauthorized	Missing/empty token
403	Forbidden	Token sent wrong way (Cookie vs Bearer)
404	Not Found	Empty path variable / wrong URL
415	Unsupported Media Type	Missing Content-Type: application/json
500	Server Error	Bad payload or real server bug

Load-Test Metrics Cheat Sheet

Metric	Read it as
Throughput	Requests/sec the server handled
Average	Mean response time (can hide outliers)
p90 / p95 / p99	90/95/99% were faster than this — report these
Latency	Time to first byte
Error %	Keep near 0

Metric **Read it as**

Saturation point Where throughput flattens but response time climbs

The Golden Rules

1. **Build in GUI, run for load in Non-GUI.**
2. **Never run a real load test with View Results Tree enabled.**
3. **Assert the body, not just the status code.**
4. **Correlate every dynamic value** (tokens, IDs) — never hardcode.
5. **Parameterize with -J** so one .jmx serves every environment and load level.
6. **Report percentiles**, not just averages.
7. **Name your samplers** by business action — the report depends on it.
8. **A Debug Sampler answers "is my variable set?"** faster than anything else.

Workshop Flow Summary

Section	Duration	What You Do
1. Install & Launch	15 min	Install, GUI vs Non-GUI, CLI options, open/save .jmx
2. Fundamentals	20 min	The tree, every component, execution order
3. First Test	20 min	Google test: Thread Group → Request → Assertion → Listener
4. API Testing	30 min	GET/POST/PUT/PATCH/DELETE, headers, body, auth
5. Postman → JMeter Map	10 min	Translate everything you already know
6. Variables & Env	20 min	UDV, properties, -J, DEV/QA/PROD config
7. Response Validation	20 min	Codes, body, JSON, headers, time, extractors

Section	Duration	What You Do
8. Extract Multiple Values	15 min	Multi-field extraction + Debug Sampler
9. Request Chaining	20 min	Token + ID chain across 5 requests
10. JSR223 & Groovy	25 min	Scripting, vars/props/ctx/prev/log
11. Dummy Sampler	10 min	Fake responses, debug data flow
12. Data-Driven CSV	20 min	CSV Data Set Config, sharing, EOF
13. Dynamic Data	15 min	Functions + Groovy for unique data
14. Assertions	15 min	Every assertion type
15. Controllers	20 min	Once Only, If, Transaction, Throughput...
16. Timers	10 min	Think-time & throughput shaping
17. Correlation	20 min	Dynamic values done right
18. Load Testing Concepts	15 min	Types & metrics vocabulary
19. Load Test Example	20 min	100 users, then scale 10→200
20. Results & Reporting	15 min	Listeners & which to avoid
21. HTML Dashboard	15 min	-e -o report, interpret graphs
22. GUI vs Non-GUI	10 min	Best practices, the golden rule
23. Debugging	20 min	Symptom → cause → fix
24. Mini Project	40 min	Full chain, then convert to load test
Total	~7 hrs of material	Trim optional sections to fit a 4–5 hr session

Fitting 4–5 hours: the core path is Sections 1–4, 6–9, 12, 14, and 24 (functional API testing) plus a light pass through 18–22 (load testing). Sections 10, 11, 13, 15–17 make excellent deeper-dive or take-home material.

Built as the JMeter companion to the QA Automation (Postman) Workshop. Build in GUI, run headless, correlate everything, and always ask not just "does it work?" but "does it work under load?"